Dropping a Quantum Temporary

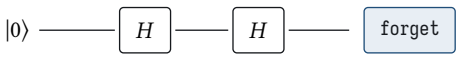
ANONYMOUS AUTHOR(S)

CCS Concepts: • **Software and its engineering** → **Formal language definitions**; • **Theory of computation** → *Programming logic*.

Additional Key Words and Phrases: quantum programming languages, uncomputation, ancilla, deallocation, dynamic lifting, type systems, contextual equivalence

1 Introduction

Here is a simple two-gate quantum circuit. It allocates a qubit, applies a Hadamard, applies a second Hadamard, and asks the compiler to forget it:



$H \cdot H = I$: the register is $|0\rangle$ again

Forget here means something specific, and it is the subject of the paper: we want the qubit to go back to a free pool, and a later allocation should be able to *reuse* it without reinitialising it. Qubits are the scarce resource. A compiler that cannot reuse a temporary allocates a fresh one, and the width of the program grows with every scratch value it ever computed. The established route to reuse is *uncomputation*: run the computation that produced the scratch value backwards, so that the qubit is provably back at $|0\rangle$, and hand it to the pool clean. That is one way to make reuse safe. This paper asks what reuse actually requires, and the answer is, perhaps surprisingly, weaker than restoration: the qubit need not be back at $|0\rangle$, but it must carry no information the rest of the program can read. What we certify is the state in which a temporary may be released; the allocator itself, and what it means to hand the same qubit out again under a measurement record, are outside the model.

Every classical model of computation throws things away for free—a Turing machine overwrites its tape, a β-reduction discards its redex, a stack pops—because overwriting is a primitive irreversible operation there, and the classical alternatives it hides cannot interfere. Bennett's reversible machine buys the discarded value back, ending with its output beside "a reconstructed copy of the input" [2]. A superposition of control withdraws the licence. The temporary is a tensor factor of the state space and not one alternative among several, so discarding it is not the choosing of a branch: the joint state pairs each temporary basis vector with a visible vector of its own, and the inner products among those decide what interference survives. That is why *forget* has content here, and why its condition cannot be read off how the value was produced.

In Silq [3] the circuit is the program:

```
t0 := 0:B;
t1 := H(t0);
t := H(t1);
forget(t);
```

The register is $|0\rangle$: two Hadamards compose to the identity. Discarding it is safe, and Silq agrees that it is safe if we use the variant `forget(t=0:B)`. The automatic `forget`, however, is refused with error: *cannot synthesize forget expression.*

The refusal is not really a bug: Silq's rule asks how the value was *produced*, not what the register *is*. For Silq, H is not `qfree`, which is Silq's word for an operation whose semantics "can be described classically" and which never creates superposition [3, §4.1], so no uncomputation can be synthesised for `t`, and the automatic `forget` is refused. Unqomp refuses it as well for the same reason: it synthesises uncomputation rather than checking it, and its correctness section shows why a non-`qfree` gate on

an ancilla defeats the mechanism [31]. The other kind of existing check accepts this program, for an instructive reason. Q# requires a released qubit to be back in the state it was allocated in [30, 39], and Quipper's `with_ancilla` closes on the programmer's promise of the same [15]. Here the register *is* back at $|0\rangle$. Program (a) below is where those rules refuse a safe drop. Every rule of either kind draws a line through the safe programs, and the systems that publish their rule say so: ReQWIRE calls its syntactic conditions "sufficient but not necessary" [34], and Qutes calls its own detection "necessarily incomplete" [10]. Silq is the one we ran, and section 11 compares the rest.

So the gap is known and in print. *The gap:* a drop is safe when no continuation of the program can observe it, that is, when the temporary carries no information about the visible register at the moment it is dropped. The existing checks ask about the program's syntax instead—how the value was produced, whether the register is back where it started, who owns it and for how long—and section 2 sets the three against each other. Neither of the first two is complete, and the two programs above are the witnesses: the two-gate circuit is a safe drop the producer check refuses, and program (a) is a safe drop the restoration check refuses. The recent tools decide a neighbouring predicate at circuit level: whether a circuit restores its ancilla [27], whether a restoring uncomputation exists [28], and how to schedule borrows given blocks already safe [47]. Drop safety as we formulate it here lies strictly between restoration and restorability. Call a program *restored* when it itself returns the temporary to the state it was allocated in, and *restorable* when some continuation could [28]. In the unrestricted measurement-free calculus *restored* implies *safe to drop* implies *restorable*, and neither converse holds: program (a) is safe and not restored, and Bennett's compute–copy–uncompute, with its target counted into the temporary, is restorable and, unless its circuit is constant, not safe (section 5.1). Over a fixed coefficient ring the second implication asks for more—the splitting condition of section 5, and over a finite repertoire an exact-synthesis step besides. None of those three tools reads a program with measurement, where the verdict may depend on the measurement record (section 7). We ask two questions about the gap. *What is the condition, stated as a property of programs rather than of matrices?* And *can the gap be closed at all?* The first has an exact answer. The second has an exact answer too, and it is negative—but only past a boundary we can draw, and the negative has a type that the natural way of stating it gets wrong.

Three programs

We work in a small language with one scoped temporary t, one visible register v, and an affine `drop` whose typing rule carries no side condition. A side condition would have to say *this temporary is uncorrelated from* v, a fact about what the program computes, and a rule that demanded it would make the unsafe programs untypable, so the checks that refuse them could not be compared.

The *temporary* t is the ancilla of the quantum computing literature: a register allocated inside a scope and dropped at its end. The *visible* register v is everything else in scope—the data qubits—and it is called visible because it is what the rest of the program can see after the drop. Both are *registers* of arbitrary finite dimension, so "one temporary and one visible register" is not two qubits unless the dimension is fixed at two. An n-qubit workspace is a visible register of dimension 2^n, and several temporaries in nested scopes compose into one big temporary of product dimension with the verdict preserved exactly (section 9). The results therefore apply to programs of any width, and section 10 runs the checker on eight textbook algorithms. Section 5 is the one section needing a ring-theoretic input, the column lemmas of the exact-synthesis literature, and it says which rings have them. One temporary is the smallest system in which the question has content: without the visible register there is no observer, and without joint gates there is no channel. An impossibility is only as strong as its hypotheses are weak, so the negatives are proved there on purpose.

Each program below allocates the temporary, computes, and drops it. A run leaves the temporary *constant* when it leaves it in one fixed state independent of the input.

Example 1.1 (the running examples).

(a) *superpose, then drop*	(b) *superpose, then read*	(c) *measure, then read*
`let t = new |0>`	`let t = new |0>`	`let t = new |0>`
`  t := H t;`	`  t := H t;`	`  t := tilt t;`
		`  let p = lift(meas t in S)`
	`  v := v + t;`	`  v := v + t;`
`  drop t; v`	`  drop t; v`	`  drop t; v`

Program (a) superposes the temporary and drops it. Its final state is $|+\rangle$ whatever the input: constant, uncorrelated with v, and not $|0\rangle$.

Program (b) reads the temporary into v before dropping it. It is the one-line proof that the affine rule cannot be silent: after the read, v and t are entangled, and forgetting t decoheres v.

A gate in this language is any orthogonal matrix (definition 3.1), and the $tilt$ gate in program (c) is $tilt = \frac{1}{3}\left(\begin{smallmatrix} 2 & 2 & 1 \\ 2 & -1 & -2 \\ 1 & -2 & 2 \end{smallmatrix}\right)$, with $tilt\,|0\rangle = \frac{1}{3}(2, 2, 1)$.

Program (c) measures the temporary, then reads it into v and drops it, and no gate after the measurement branches on the outcome. The gate $tilt$ is an orthogonal gate on a three-dimensional temporary, chosen so that the measured block $S = \{0, 1\}$ and its complement both survive with nonzero weight. Which block survives is decided at run time. On one outcome the drop is safe, on the other it is not, and nothing in the program's syntax says which run is happening.

Why discard is not free here

In some languages this question has no content. In the quantum λ-calculus quantum data is affine and discard is a typing rule that asks nothing [35], because discard is a partial trace over a density-matrix semantics: the continuation receives the reduced state of the registers it can still name, which is correct whatever the discarded register held, and that register is never reused. Here we want it reused. The compiler traces nothing, and it hands the same physical qubit to the next allocation, on the promise that the next allocation will find nothing in it. Section 3 makes the promise precise: Drop has no side condition, the elaboration of `drop` is the identity, and the result type of a program, which no longer mentions the temporary, asserts that no continuation will look at it. So the promise is the affine rule's *premise*, not its conclusion, and the rule leaves it unchecked on purpose: the programs that break it must be expressible, as program (b) of example 1.1 is, well typed with a wrong drop.

What a compiler gets, and what it pays

The criterion is *stated* on programs and *decided* on matrices, and the two are not in tension. Stated: a drop is safe when the program is equivalent to one that never touched the temporary, a sentence with no matrix in it (section 4). Decided: the program text names its gates, each gate is a matrix, and multiplying them in the order the text gives yields the denotation K (section 3). The decision then is a rank test on K, static, needing neither an input nor a run. The price is a proportionality test on the d Kraus operators, each an $m \times m$ matrix: polynomial in the joint dimension dm, hence exponential in the qubits in scope, the cost of writing the denotation down at all. Once the verdict depends on which outcome occurred, one verdict for all records is wrong on one of them, and a sound one is wrong by refusing a safe run (section 7). From section 8 a compiler gets instead a table with one row per measurement record, at most 2^ℓ for ℓ lifted measurements on the command's longest path, each row one proportionality test, computed before the run and consulted at run time by a lookup on the classical outcomes, and ℓ is at most two in the programs of section 10.

Contributions

- **Measurement-free: an exact criterion on the program text** (section 4). Three conditions on a typed program coincide: it is *weakenable*, some typed program that prepares the temporary once and never touches it again having the same denotation; it is *drop-safe*, its temporary ending in one fixed state whatever the input; and no continuation can distinguish it from a program of that shape (theorems 4.2 and 4.3). All three say the channel the drop induces has Kraus rank one (theorem 6.2), a finite computation on the matrix the program's gates multiply out to. The rank-one arithmetic is textbook—the Choi rank of an *isometric channel* [9, 25, 45]—and what is ours is that it is asked of a program's denotation. Restoring the temporary to where it was allocated is this criterion pinned to one ray, and it refuses eight of the twelve safe catalogue programs, Deutsch–Jozsa among them.
- **With measurement: a verdict fixed before the run is incomplete** (section 7). We extend the calculus with measurement, following Proto-Quipper's *dynamic lifting* [12]; a typed program of the affine calculus elaborates to a lift-free command of it, with the same denotation. Within the extension we exhibit a well-typed command with two reachable measurement records of opposite verdict on the same drop, and no *sound* record-oblivious verdict keeps the safe one: the commands on which a safe run must be lost are exactly the outcome-dependent ones (section 7), which the presence of a lift does not characterise.
- **One exact repair: a record-indexed table** (section 8). The boundary forces record awareness—the missing bit exists only after the measurement—and one exact realisation is a table emitted before the run, one row per record and at most 2^{ℓ} rows for ℓ lifts, each row the proportionality test of section 4: the static analysis's own cost, once per record. The table is exact on the very command where a sound record-oblivious verdict must lose a safe run.

Outline Sections 5 and 5.1, in summary. Over a gate ring completable in the temporary's dimension the criterion acquires one side condition, that the projector onto the final state factor as $zz^{\dagger}$ over the ring: automatic over the Clifford+T ring, false over its real subring. Deciding is coNP-complete already on permutation programs.

Outline Sections 9 and 10, in summary. Composition reduces several temporaries to one, though dropping them one at a time can cost a square root; typing splits per lifetime and drop safety does not. Two decision procedures sharing no code but one arithmetic backend agree on every catalogue verdict.

2 The idea, on three programs

This section carries the three programs of example 1.1 through the whole paper informally.

2.1 The temporary is the scrutinee of an invisible switch

From a language implementation perspective the whole idea is that a quantum program with a temporary of dimension d is a `switch` with d branches. The scrutinee is the temporary's basis state: the ith branch is what the program does to the visible register when the temporary is in state $|i\rangle$. Call that branch's action K_i. It is an $m \times m$ matrix on the visible register, read off the program's denotation K as the block of rows labelled i:

$$Kv \;=\; \sum_{i=0}^{d-1} |i\rangle \otimes K_i v, \qquad K_i = (\langle i| \otimes I)\, K.$$

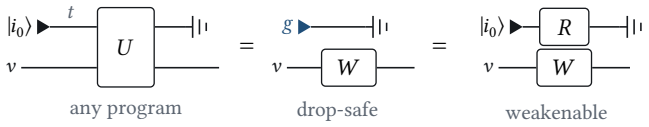


Fig. 1. The criterion. A triangle starts a register in a fixed state and the bars end one, which is `drop`.
Left: any program of the calculus.
Middle: the drop-safe shape (definition 3.2): the temporary comes out in one state g whatever v was, and nothing crosses between the registers.
Right: the weakenable shape (definition 4.1): the temporary is prepared by one gate R and dropped, never having met v.
The left program is drop-safe exactly when it equals the middle, and theorem 4.2 says that is exactly when it equals the right.

The K_i are the *Kraus operators* [25] of the channel that dropping the temporary induces on the visible register: tracing the temporary out of $K\rho K^\dagger$ leaves $\rho \mapsto \sum_i K_i \rho K_i^\dagger$. Quantum mechanics adds one thing to the switch: every branch runs, and the results are kept side by side, each labelled by its branch. The label captures the difficulty: the temporary holds a memory of what happened to the visible register, and a memory is exactly what a drop must not throw away while anyone downstream can read it.

Now `drop` t deletes the scrutinee, and the question is whether anybody downstream can tell. If every branch does the same thing, which in matrices means $K_i = g_i W$, one visible action W scaled differently per branch, then the switch was fake: the program's behaviour never depended on the scrutinee, deleting it changes nothing, and thus the drop is safe. If two branches do *different* things, the switch was real: the downstream code behaves one way when the temporary went left and another when it went right, so the temporary recorded something about the visible input, and deleting it destroys a correlation that was there. So the safety of a drop is a proportionality condition: the Kraus operators $K_0, \ldots, K_{d-1}$ are pairwise proportional, all scalar multiples of one isometry W. Equivalently, the channel the program induces has *Kraus rank one*: it is conjugation by that one isometry. On the matrix a program's gates multiply out to, that is a finite calculation in exact arithmetic, so the verdict is decided and not estimated.

The arithmetic is not new. That a channel is $\rho \mapsto \sum_i K_i \rho K_i^\dagger$, that two families of operators describe one channel exactly when an isometry relates them—a unitary, once the shorter family is padded with zeros—and that the fewest a channel needs is the rank of its Choi matrix, the *Choi rank*, are the standard structure theory of channels [9, 25, 45], and rank one is its smallest case, the *isometric channel*. What is new is what the rank is asked about: the denotation of a *program*.

2.2 Program (a) is safe, and it does not restore anything

Program (a) applies H to the temporary and nothing else. Program (b) then reads the temporary into v by CNOT. Write each denotation as a matrix, rows indexed by the temporary's level first, so that the rows at level 0 form K_0 and those at level 1 form K_1:

Program (a) sends v to $\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \otimes v$:

$$K_{(a)} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 0 & 1 \end{pmatrix} \begin{matrix} \} K_0 = \frac{1}{\sqrt{2}} I \\ \} K_1 = \frac{1}{\sqrt{2}} I \end{matrix}$$

One visible action, scaled the same on both branches: the switch is fake and the drop is safe, though the temporary ends in $|+\rangle$, not where it started.

Program (b) sends v to $\frac{1}{\sqrt{2}}(|0\rangle \otimes v + |1\rangle \otimes Xv)$:

$$K_{(b)} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 1 \\ 1 & 0 \end{pmatrix} \begin{matrix} \} K_0 = \frac{1}{\sqrt{2}} I \\ \} K_1 = \frac{1}{\sqrt{2}} X \end{matrix}$$

Two different visible actions. The switch is real, and the drop is wrong.

The checker of section 10 also forms the Gram matrix $\Gamma_{ij} = \mathrm{tr}(K_i^\dagger K_j)$ (proposition 3.4): $\left(\begin{smallmatrix} 1 & 1 \\ 1 & 1 \end{smallmatrix}\right)$ of rank one for (a), the two rows equal, and $\left(\begin{smallmatrix} 1 & 0 \\ 0 & 1 \end{smallmatrix}\right)$ of rank two for (b). The channel is what the rest of the program sees: after the drop the visible register carries $\sum_i K_i \rho K_i^\dagger$ and nothing else. For program (a) that is ρ, the input untouched. For program (b) it is $\frac{1}{2}\rho + \frac{1}{2}X\rho X$, a bit flip with probability one half, which a continuation that reads v can tell.

When we say *the temporary is left constant* we do not mean untouched: program (a) moves the temporary to $|+\rangle$, for every input. Nor do we mean *a product at each input*: the copy $t := t \oplus v$ leaves a product on $|0\rangle$ and on $|1\rangle$ and a Bell state on $|+\rangle$, and even a product at *every* input admits programs the definition excludes, the swap of t and v among them (proposition 3.3).

Section 4 says the same thing in the words a language can use. Program (a) is *weakenable*: weakening is the structural rule that lets a hypothesis go unused, and program (a) has the denotation of a program whose temporary is unused, the right-hand picture of fig. 1 with $R = H$ and $W = I$. The equivalence is contextual in the sense familiar from the λ-calculus: the contexts are the continuations of the calculus, a gate on the visible register after the scope followed by a measurement, and a program is weakenable exactly when no continuation distinguishes it from some program of that shape (theorem 4.3). So *this temporary may be dropped* and *no continuation can tell* are one statement, and the proportionality test decides it.

2.3 What the existing checks see

We encountered three kinds of check in the literature, and each asks a question about the syntax. A *producer-based* check—Silq's `forget` and Unqomp's synthesised uncomputation [3, 31]—asks how the value was produced: only values produced classically may be forgotten. A *restore-based* check—Q# at release and Quipper's `with_ancilla` promise [15, 30, 39]—asks whether the register is back in the state it was allocated in. A *lifetime* check—Cavy, Qurts and Qutes [10, 18, 29]—asks who owns the register and how long it lives, and enforces or synthesises restoration at the boundary; it answers a different question, whether cleanup can be inserted, not whether the temporary may be dropped in the state the program leaves it in. The first two miss different programs. A producer check refuses a value produced non-classically that is uncorrelated by the time it is dropped, which is program (a); a restore check refuses every program leaving the temporary constant anywhere but $|0\rangle$, however produced. Safety is about *correlation*, not *location*. Deutsch–Jozsa is refused by both: its kickback ancilla is prepared in $|-\rangle$ by a non-classical gate, used, and left there (section 4.2).

2.4 Program (c): one text, two verdicts

Program (c) measures the temporary and then reads it, and no later gate branches on the outcome—yet the verdict does. Figure 2 is the whole difficulty in one picture: one program, two reachable outcomes, two verdicts. Now ask what a record-oblivious verdict—a function from commands to booleans—can do with its one boolean. If it says *yes*, it has accepted a drop that is wrong on the

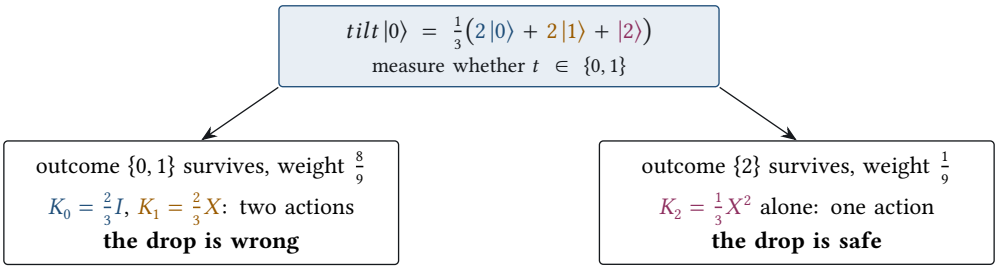


Fig. 2. Program (c). The allocated state is tilted so that it has a foot in the measured block and a foot outside it. Reading the program determines the verdict on each foot: refuse if the block survives, admit if its complement does. What reading the program cannot determine is which foot survives: the measurement decides that when the program runs, dynamic lifting hands that one bit to the compiled code, and the check of section 8 reads it there. X is the shift $|j\rangle \mapsto |j+1\rangle$ the read applies to the visible register.

branch of weight $8/9$, so it is unsound. If it says *no*, it has refused a drop that is safe on the branch of weight $1/9$, so it is incomplete there. Nothing about the syntax distinguishes the two branches, and section 7 makes that a theorem with a quantifier on it (section 7).

Nothing here runs a program. On a measurement-free scope the verdict is a function of the program text alone; after a measurement the missing argument is which outcomes survived, not an input. Reading the program fixes the verdict on every outcome, but not which outcome the run is on, and statistics are the wrong instrument for the same reason (section 6.1).

2.5 The check at the lift

The shape of the failure dictates the shape of the fix. A static analysis must emit its boolean before the run, but the deciding fact does not exist until the run, after the measurement. So we emit instead a predicate on the classical outcome, whose table the compiler computes before the run and consults at the *lift*, where a measurement outcome becomes available to compiled code. In plain words: the compiler decides the drop for every outcome the measurement could have, and the run looks up the one that happened. That outcome is all the run contributes. For program (c) it is one classical bit, whether the block S survived, and it selects the row. Section 8 will prove it sound and complete where every static analysis is not, at one proportionality test per record.

> *A drop is safe when the rest of the program cannot tell: before measurement the program decides that exactly, by a proportionality test on its Kraus operators. After measurement the program is exactly one argument short—the measurement record—and a check at the lift supplies it.*

3 The calculus

Section 2 was informal. This section fixes the language the theorems are about. Registers are real here and in section 4, so $\top$ is the $\dagger$ of the statements: drop safety is a proportionality condition and does not see the field. Section 5, where phase is the content, is complex throughout.

Definition 3.1 (the affine calculus). Register and value types, and terms, are

$$Q ::= \mathsf{tmp}[d] \mid \mathsf{vis}[m], \qquad A, B ::= Q \mid A \otimes B,$$

$$e ::= v \mid \langle v_1, v_2\rangle \mid \texttt{let}\ t = \texttt{new}\ i_0\ \texttt{in}\ e \mid \texttt{let}\ (t', v') = U(t, v)\ \texttt{in}\ e \mid \texttt{drop}\ t\,;e.$$

Contexts are finite partial maps from register names to register types, and the typing rules are

$$\frac{}{v \,:\, A \vdash v \,:\, A}\ \textsc{Var} \qquad \frac{v_1 \neq v_2}{v_1 \,:\, A,\, v_2 \,:\, B \vdash \langle v_1, v_2 \rangle \,:\, A \otimes B}\ \textsc{Pair}$$

$$\frac{t \notin \mathrm{dom}(\Delta) \quad i_0 \in [d] \quad \mathsf{allocFree}(e) \quad \Delta,\, t \,:\, \mathsf{tmp}[d] \vdash e \,:\, C}{\Delta \vdash \mathtt{let}\ t = \mathtt{new}\ i_0\ \mathtt{in}\ e \,:\, C}\ \textsc{New}$$

$$\frac{\begin{array}{c} t \neq v \quad t' \neq v' \\ U \,:\, \mathbb{R}^d \otimes \mathbb{R}^m \to \mathbb{R}^d \otimes \mathbb{R}^m \quad U^\top U = UU^\top = I \quad \Delta,\, t' \,:\, \mathsf{tmp}[d],\, v' \,:\, \mathsf{vis}[m] \vdash e \,:\, C \end{array}}{\Delta,\, t \,:\, \mathsf{tmp}[d],\, v \,:\, \mathsf{vis}[m] \vdash \mathtt{let}\ (t', v') = U(t, v)\ \mathtt{in}\ e \,:\, C}\ \textsc{Gate}$$

$$\frac{\Delta \vdash e \,:\, C}{\Delta,\, t \,:\, \mathsf{tmp}[d] \vdash \mathtt{drop}\ t\,;e \,:\, C}\ \textsc{Drop}$$

The binders t' and v' bind in e, and terms are identified up to α-renaming. A *program* is typed when $v \,:\, \mathsf{vis}[m] \vdash \mathtt{let}\ t = \mathtt{new}\ i_0\ \mathtt{in}\ e \,:\, \mathsf{vis}[m]$. The result type is $\mathsf{vis}[m]$ rather than $\mathsf{tmp}[d] \otimes \mathsf{vis}[m]$, because we want the type to record that the temporary was weakened away. A *gate* in Gate always acts on both registers at once. We call such a gate *joint*, and a gate of the form $R \otimes I$ or $I \otimes W$ *local*. Contexts are *affine* in both halves. There is no contraction because copying a register is not a linear map (no-cloning). There is no *implicit* weakening because implicit weakening is the entire subject of the paper: Drop is weakening made explicit, and under implicit weakening program (b) of example 1.1 would type with its temporary silently forgotten. The premise $\mathsf{allocFree}(e)$ of New—the body of an allocation contains no second allocation—gives a program exactly one temporary scope, and section 9 lifts this.

Drop has no premise about safety, and that is not an oversight: if `drop` demanded safety as a premise, unsafe programs such as program (b) would not be *expressible*, and the predicates that reject them could not be compared. **The result type** $\mathsf{vis}[m]$ **is a promise, not a guarantee.** The elaboration of `drop` is the identity, the temporary survives as a latent tensor factor, and the type asserts that no continuation will look at it. When the drop is unsafe the type is wrong about the denotation, and detecting exactly that is the rest of the paper. Typing is algorithmic and types are unique, the outermost constructor of e selecting one rule whose premises are decided from Δ and e. The calculus admits an arbitrary orthogonal matrix as a gate, and section 5 says what a *finite* gate set costs on top of that.

3.1 The denotation, and drop safety

The rules say which programs exist. This subsection says what a program computes, and when its drop is safe. The temporary has state space $\mathbb{R}^d$ and the visible register $\mathbb{R}^m$, with $m \geq 1$ throughout. A typed program elaborates to a straight-line list of gates by recursion on the term: allocation contributes the isometry $v \mapsto e_{i_0} \otimes v$, a gate contributes its matrix, and `drop` contributes nothing:

$$[\![\mathtt{let}\ t = \mathtt{new}\ i_0\ \mathtt{in}\ e]\!] = [\![e]\!] \circ (v \mapsto e_{i_0} \otimes v), \qquad [\![\mathtt{let}\ (t', v') = U(t, v)\ \mathtt{in}\ e]\!] = [\![e]\!] \circ U,$$

$$[\![\mathtt{drop}\ t\,;e]\!] = [\![e]\!], \qquad [\![v]\!] = I, \qquad [\![\langle v_1, v_2 \rangle]\!] = \Pi_{v_1 v_2},$$

where $\Pi_{v_1 v_2}$ permutes the context's tensor factors to put v_1 first and v_2 second, and is the identity when they already stand that way. The *denotation* K of a typed program is this composite, a linear

map $\mathbb{R}^m \to \mathbb{R}^d \otimes \mathbb{R}^m$ computed from the program text alone by multiplying the matrices of the gates it names. It is an isometry, being an isometric allocation followed by unitaries. Norm preservation is the realizability fact typing supplies; absence of leakage is the rank-one condition below, and the two are different. (Section 6 adds measurement, and theorem 6.1 there says that this same matrix is what its machine computes.)

Definition 3.2 (Kraus operators, drop safety). For a linear map $K : \mathbb{R}^m \to \mathbb{R}^d \otimes \mathbb{R}^m$, its *Kraus operators* are the $m \times m$ blocks $K_i = (\langle i| \otimes I)\, K$, so that $Kv = \sum_i |i\rangle \otimes K_i v$. The map K is *drop-safe* when $K = g \otimes W$ for a vector $g \in \mathbb{R}^d$ and a matrix W, that is, $Kv = g \otimes Wv$ for every v. A program is drop-safe when its denotation is.

The name *Kraus operators* is standard, and the channel is why. Discarding the temporary leaves the visible register in

$$\Phi(\rho) \;=\; \mathrm{Tr}_{\mathrm{tmp}}\big(K \rho K^{\top}\big) \;=\; \sum_i K_i \rho K_i^{\top},$$

the second equality by expanding the partial trace in the basis $|i\rangle$. That K is an isometry is exactly $\sum_i K_i^{\top} K_i = I$, which makes Φ trace-preserving; over $\mathbb{C}$ every completely positive map has this form, and every such sum is completely positive [9, 25]. Read backwards, K is a dilation of Φ whose environment is the temporary, and the drop is the partial trace. The family is not unique—mixing by an isometry, $K'_j = \sum_i u_{ji} K_i$, gives the same channel—so what belongs to the channel is how many independent operators it has, not which they are, and that is the number the criterion reads.

Drop safety says the temporary is *constant* at the drop: it ends in the one state g whatever the input, unentangled from the visible register. The force of the definition is the quantifier order, one g for *all* v. Four forms are equivalent, and each is used below: (1) $K = g \otimes W$; (2) the Kraus operators are pairwise proportional, $K_i = g_i W$; (3) every 2×2 minor of the $d \times m^2$ matrix $(i, (b, c)) \mapsto K_{(i,b),c}$ vanishes; (4) the Gram matrix $\Gamma_{ij} = \mathrm{tr}(K_i^{\top} K_j)$ of the Kraus operators has rank at most one. Form (3) is a finite computation on exact numbers, valid over any field, and section 5.1 prices it. Form (4) is form (3) compressed to $d \times d$: $\Gamma = MM^{\top}$ for the matrix M of form (3), a Gram matrix has the rank of the family it is built from, and it is the form the checker cross-checks against. Since K is an isometry, $\mathrm{tr}\,\Gamma = \sum_i \mathrm{tr}(K_i^{\top} K_i) = m$, and a positive semidefinite matrix of trace m has rank at most one exactly when its eigenvalues are m once and 0 otherwise, that is, when

$$\Gamma^2 = m\,\Gamma, \qquad \text{equivalently} \qquad \left(\frac{\Gamma}{m}\right)^2 = \frac{\Gamma}{m}\ :$$

the drop is safe exactly when Γ/m is *idempotent*, a rank-one projector, and then it is $gg^{\top}$ for the unit final state g, since $\mathrm{tr}(W^{\top} W) = m$ for the isometry W. That projector is the temporary's own state at the maximally mixed visible input (Appendix A.5), and a state is idempotent exactly when it is pure. So the criterion has a one-line reading on the temporary alone: *a drop is safe exactly when the temporary comes out pure at the maximally mixed visible input.* That reference input is not a convenience, as the next proposition shows. The pair (g, W) is determined up to $(\lambda g, \lambda^{-1} W)$, so "unit g" is a choice of representative; the zero map is drop-safe.

Here is why that single reference input is enough, and why the obvious alternative is not the same test. A reader meeting definition 3.2 may well ask what is wrong with checking the joint state *at each input separately*: run the program on v, see whether what comes out is a product of a temporary state with a visible state, and repeat for every v. That is a weaker demand—it lets the temporary's state depend on v, where drop safety does not—and it is weak enough to admit programs that leak. The next proposition says exactly how much weaker: pointwise productness has only two solutions, and only the first of them is a safe drop.

In revision

Proposition 3.3 (a product at every input). *Let $K : \mathbb{R}^m \to \mathbb{R}^d \otimes \mathbb{R}^m$ be linear, and suppose Kv is a product vector, or zero, for every v. Then K has one of exactly two shapes:*

the temporary is fixed $K = g \otimes W$ for a vector $g \in \mathbb{R}^d$ and a matrix W on $\mathbb{R}^m$ — the visible register varies, the temporary does not;

the visible register is fixed $Kv = Gv \otimes w$ for a fixed nonzero $w \in \mathbb{R}^m$ and a linear $G : \mathbb{R}^m \to \mathbb{R}^d$ — the temporary varies, the visible register does not.

The first shape is drop safety. A map of the second is drop-safe exactly when G has rank at most one, so such an isometry with $m \geq 2$ never is.

The proof (Appendix A.1) is linearity: two inputs whose temporary factors are not parallel force, through $K(v_1 + v_2)$, their visible factors to be parallel—so one of the two factors has to stand still, and which one it is gives the two cases. The second case is the leak. It puts the input into the temporary and leaves the visible register alone, and the plainest instance is the **swap** of t and v at $d = m$: it is a product at every input, its Kraus operators are $e_{i_0} e_i^\top$, and unsafe for every $m \geq 2$, having moved the whole input into the register about to be discarded. So *a product at every input* admits programs definition 3.2 excludes, and the quantifier order—one g for *all* v—rules them out.

Proposition 3.4 (what the Gram matrix is). *Let K be the denotation of a typed program, with Kraus operators K_i and Gram matrix Γ.*

(1) The drop is safe if and only if $K_i^\dagger K_j = c_{ij} I$ for all i, j, and then $c = \Gamma/m$; over $\mathbb{C}$, if and only if the temporary's final state is the same at every unit input.

(2) $\operatorname{rank}\Gamma$ *is the least temporary dimension of a typed program whose visible channel $\rho \mapsto \sum_i K_i \rho K_i^\dagger$ is that of K.*

(3) Γ is unchanged by a visible gate after the scope, and a gate U on the temporary carries it to $\bar{U}\Gamma U^\top$.

The proof (Appendix A.1) is short. Part (1) is the Knill–Laflamme condition [21] with the whole visible register as the code, the K_i as the errors and the temporary as the environment, which therefore learns nothing [26]. Its second half is noninterference [14, 48], over $\mathbb{C}$ only—a real rotation refutes it (Appendix A.1)—and the impurity ι of section 5.1 is its quantitative degree. Part (2) reads Choi's rank [9] as a resource, the least width a temporary needs for the observed visible behaviour. Part (3) makes Γ a certificate: nothing the continuation does changes it.

One lemma about that family is used twice below, once for the criterion of section 4 and once for its reading as a channel, so it is stated here where the Kraus operators are introduced.

Lemma 3.5 (proportionality). *Let $K_0, \ldots, K_{d-1}$ and M be linear maps on $\mathbb{R}^m$, and suppose $\sum_i (K_i x)(K_i x)^\top = (Mx)(Mx)^\top$ for every x. Then $K_i = c_i M$ for scalars c_i.*

The proof (Appendix A.1) reads the hypothesis one matrix entry at a time, so it is worth writing the two sides out. Both are $m \times m$ and symmetric; at $m = 2$, abbreviating $u_i = K_i x$ and $z = Mx$, the hypothesis says

$$\sum_i \begin{pmatrix} (u_i)_0^2 & (u_i)_0 (u_i)_1 \\ (u_i)_1 (u_i)_0 & (u_i)_1^2 \end{pmatrix} = \begin{pmatrix} z_0^2 & z_0 z_1 \\ z_1 z_0 & z_1^2 \end{pmatrix}.$$

The coloured diagonal settles each row on its own. Where $z_b = 0$ a sum of squares is zero, so every $(u_i)_b$ vanishes with it; where $z_b \neq 0$ the ratios $(u_i)_b / z_b$ form a unit vector $c^{(b)}$ indexed by i. That is

the sense in which each row of each K_i is a multiple of the corresponding row of M. The uncoloured off-diagonal stops the multiples drifting from row to row: it reads $\langle c^{(0)}, c^{(1)} \rangle = 1$, and two unit vectors with inner product one are equal, so the rows share one c: the statement's c_i.

3.2 The constructs in existing systems

Every construct in the calculus is one that existing systems already have. `new` is Silq's local quantum variable, Q#'s `use`, and Quipper's `with_ancilla`; Q#'s `borrow` is the dirty variant, which carries no initialisation promise. `drop` is Silq's `forget`, the end of a Q# `use` block, and the close of `with_ancilla`. The side condition is Silq's `qfree` producer, checked by its type checker, Q#'s requirement that a qubit be released in the state it was allocated in, checked at run time on simulators, and in Quipper the programmer's promise, checked by nothing. The `lift(meas)` of section 6 is Q#'s `M` followed by classical control, and Quipper's dynamic lifting. The one mismatch runs the other way: Silq's `measure` consumes the register, where a lift leaves the temporary live and restricted to the surviving block. Section 10 says which of these are transcripts and which documentation.

4 The criterion

Both of the paper's negatives are stated against a positive, and this is it. For the measurement-free calculus we can say exactly when `drop` is safe, and say it not as "the denotation factors"—definition 3.2, quantified over matrices—but as a statement quantified over *programs*.

Definition 4.1 (weakening witness). Write $\mathrm{sep}(R, W)$ for the term

$$\texttt{let } (t_1, v_1) = (R \otimes 1)(t, v) \texttt{ in let } (t_2, v_2) = (1 \otimes W)(t_1, v_1) \texttt{ in drop } t_2 \texttt{ ; } v_2$$

— *prepare the temporary once, compute on the visible register, drop.* A program P is *weakenable* when some *typed* program of that shape, with the same allocation, has the same denotation.

Two readings of that last sentence have to be kept apart, and the section's whole content sits in the gap between them.

being $\mathrm{sep}(R, W)$ is a fact about the program *text*: this text has one gate on the temporary, then one on the visible register, and nothing joint anywhere.

being weakenable is a fact about the *denotation*: somewhere in the calculus there is a text of that shape computing the same function—whether or not our program looks remotely like it.

The second is by far the weaker demand, and Deutsch–Jozsa is the case to hold on to. Its text has the joint gate U_f sitting in the middle of it and resembles $\mathrm{sep}(R, W)$ not at all; yet its denotation is $|-\rangle \otimes HD_fHv$ (section 4.2 works this out), and the separable program that prepares $|-\rangle$ and then applies HD_fH has exactly that denotation. The algorithm is weakenable without being separable, and by theorem 4.3 no continuation will tell the two apart. Of the artifact's seventeen catalogue programs **two** are literally of the shape; **twelve** are safe and so, by the next theorem, have a witness they need not resemble. Appendix B.1 of the supplement prints three of them beside their programs.

Theorem 4.2 (the criterion). *A typed program is weakenable if and only if it is drop-safe.*

Proof.

⟨1⟩ ($\Rightarrow$) A witness $\mathrm{sep}(R, W)$ sends $e_{i_0} \otimes v \mapsto Re_{i_0} \otimes v \mapsto Re_{i_0} \otimes Wv$, which is $g \otimes W$ with $g = Re_{i_0}$, and `drop` changes nothing.

⟨2⟩ ($\Leftarrow$) Let P be typed with denotation $Kv = g \otimes Wv$. Since K is an isometry, $\|g\| \, \|Wv\| = \|v\|$ for every v. Rescale to $\|g\| = 1$, and W is an isometry of $\mathbb{R}^m$, hence orthogonal, hence a legal gate.

⟨3⟩ It remains to prepare g from e_{i_0} by one gate. If $g = e_{i_0}$ take $R = I$. Otherwise put $w = e_{i_0} - g$ and let $R = I - 2ww^\top/\|w\|^2$, the reflection through the hyperplane $w^\perp$: it is orthogonal, and since $\langle w, e_{i_0}\rangle = 1 - \langle g, e_{i_0}\rangle = \frac{1}{2}\|w\|^2$ it sends e_{i_0} to $e_{i_0} - w = g$. Then $\mathrm{sep}(R, W)$ is typed, has the same allocation, and denotes K. □

Step ⟨3⟩ is the substance: every unit final state is preparable by one reflection or the identity, hence one Gate, so the certificate (g, W) is realised by a program, the step a finite gate set threatens (section 5). Step ⟨2⟩ is the whole contribution of the type system: its gates are unitary, so K is an isometry. Without typing that must be assumed: for arbitrary linear gates, weakenability is drop safety together with $\|Kv\| = \|v\|$ for every v, and the clause cannot be dropped, since at $d = m = 1$ the map that doubles the visible register is drop-safe and not weakenable (Appendix A.1). What is real-only here is the reflection witness; over $\mathbb{C}$ the factors are determined up to a compensating phase.

4.1 The criterion is contextual indistinguishability

Definition 4.1 compares a program with a witness at every input. That is not the same as comparing them in every *context*. A *continuation* appends one visible gate C after the scope's last gate, drops the temporary, and reads the visible register in the standard basis. What it observes at input x is the distribution $b \mapsto \sum_i \langle e_b, CK_i x\rangle^2$: the Born statistics of the visible register, temporary traced out. A class F of visible gates *distinguishes* two programs when some $C \in F$ and some input give different distributions, and a *separable program* is a typed program of the shape $\mathrm{sep}(R, W)$.

In revision

Theorem 4.3 (the criterion is contextual indistinguishability). *Let F be any class of visible gates containing the reflections. A typed program is weakenable if and only if F does not distinguish it from some separable program.*

The proof (Appendix A.1) reads the Born statistics after each reflection: agreement at every reflection says the symmetric matrices $\sum_i (K_i x)(K_i x)^\top$ and $(Wx)(Wx)^\top$ have the same quadratic form, so they coincide, and lemma 3.5 gives $K_i = c_i W$.

Three remarks. A larger class distinguishes more, so the theorem holds for all visible gates and for continuations that also touch the temporary. The class cannot be $\{I\}$: the unsafe copy $t := t \oplus v$ has the same standard-basis statistics as doing nothing at every input, since $\sum_i \langle e_b, P_i x\rangle^2 = x_b^2$ for its Kraus projectors P_i. The exact minimal class is open. And the theorem is about programs whose gates keep their register names, since the gate rule may rename its output binders.

4.2 Worked: Deutsch–Jozsa

The criterion on the textbook's first algorithm, where it says what a producer-based check cannot:

```
let t = new |0> in t := X t; t := H t;     -- the ancilla is prepared in |->
v := H v; v,t := Uf(v,t); v := H v;         -- phase kickback
drop t; v                                  -- the drop under discussion
```

Take $f : \{0,1\} \to \{0,1\}$, so that $U_f|j,t\rangle = |j, t \oplus f(j)\rangle$ and $D_f = \mathrm{diag}((-1)^{f(0)}, (-1)^{f(1)})$. At an *arbitrary* input $\sum_j c_j|j\rangle$ to U_f—not only the intended one—the state after it is $\big(\sum_j (-1)^{f(j)} c_j |j\rangle\big) \otimes |-\rangle$. The temporary factor is $|-\rangle$ whatever the input, so the scope's denotation is $g \otimes Wv$ with $g = |-\rangle$ and $W = HD_fH$, which is $\pm I$ for constant f and $\pm X$ for balanced f. By theorem 4.2 the drop is safe, and by theorem 4.3 no continuation can distinguish this program from a separable one.

It is program (a) with an algorithm attached, and every rule that reads the program text refuses it, for one of the two reasons of section 2. The producer-based rules refuse it because H is not `qfree`: Silq's automatic `forget` fails in our transcripts with *cannot synthesize forget expression*, and Unqomp's synthesis is defeated by the same gate [3, 31]. The restore-based rules refuse it because $|-\rangle$ is not $|0\rangle$: Q#'s release requirement [30, 39], the promise Quipper's `with_ancilla` closes on [15], Qutes' Reclamation [10, Def. 6], and the Π family's fractional reuse rule [7]. None of these four turns on f: none of them looks at the oracle, and each refuses on the state the temporary is left on. `dqt pool` runs the last of them. The verification logics of section 11 can admit the drop, but from a separability assertion the programmer supplies and proves, not from a check on the text.

4.3 What the pool receives, and the rule it already had

> **Planned** What the free pool receives when a drop is admitted, and what a record-indexed pool would have to receive once allocation may depend on an outcome; and how the Π family's fractional reuse rule [7] reads against the criterion. Being written; nothing here is claimed.

5 What restricting the gates costs

Theorem 4.2's right-to-left direction used that *every* unit state is preparable in one gate, because definition 3.1 admits an arbitrary orthogonal U. A compiler emits words over a fixed finite repertoire, and there preparability is a real constraint: the biconditional can acquire a side condition. We state it at the *ring* the repertoire's entries generate: every word over the repertoire is a gate over that ring, so an obstruction over the ring is one for the repertoire. The condition bites over some rings, and not over Clifford+T.

Fix a *gate ring* $\mathcal{A}$: a subring of $\mathbb{C}$ closed under conjugation, the ring the gate entries generate. A gate over $\mathcal{A}$ is an $\mathcal{A}$-valued unitary. Transport back to a concrete repertoire is not automatic: it needs exact synthesis *within the witness's allocation budget*, and the multiqubit results permit a clean ancilla [1, 13, 20], so we infer no finite-repertoire corollary. Say $\mathcal{A}$ is *completable* in dimension d when every unit vector in $\mathcal{A}^d$ is a column of a unitary over $\mathcal{A}$. Throughout this section the temporary has dimension $d \geq 2$ and is allocated at $|0\rangle$, the visible register is arbitrary, and scalars are complex. When a program U is drop-safe its final state g is determined up to phase, and the object of interest is the rank-one projector $gg^\dagger$, computed directly from the entries of U without first finding g:

$$\operatorname{proj}(U)_{i\ell} \;=\; \sum_{c} U_{(i,0),(0,c)}\,\overline{U_{(\ell,0),(0,c)}}.$$

Read the four slots of $U_{(i,b),(j,c)}$ as *amplitude from temporary j, visible c, to temporary i, visible b*, and the colours say which is which. Warm is the allocation: the input temporary is pinned to 0, because that is where a program starts it. Plum is the visible index, and it is *summed*—nothing here depends on which visible basis vector went in, which is why no clause about the visible register appears in the theorem below. Blue is what the pinnings keep: the picked cells, and the two output temporary levels i and ℓ, which are the free indices—the row and column of the $d \times d$ matrix being built. The output visible index is pinned to 0 in both factors, and for a drop-safe U any other choice gives the same matrix: the entries are then $U_{(i,b),(0,c)} = g_i W_{bc}$, so the sum is $g_i\overline{g_\ell}$ times the squared norm of one row of the isometry W, which is 1. Figure 3 is that formula drawn: the two pinnings are a block column and a row inside it, and what survives is a slab against its own conjugate.

Say $\mathcal{A}$ *splits* a rank-one projection P when $P = zz^\dagger$ for a vector z over $\mathcal{A}$. The two halves of that sentence are different demands, and fig. 4 keeps them apart: P may have every entry in $\mathcal{A}$ while no vector over $\mathcal{A}$ splits it, because the splitting vector's entries are square roots of P's.

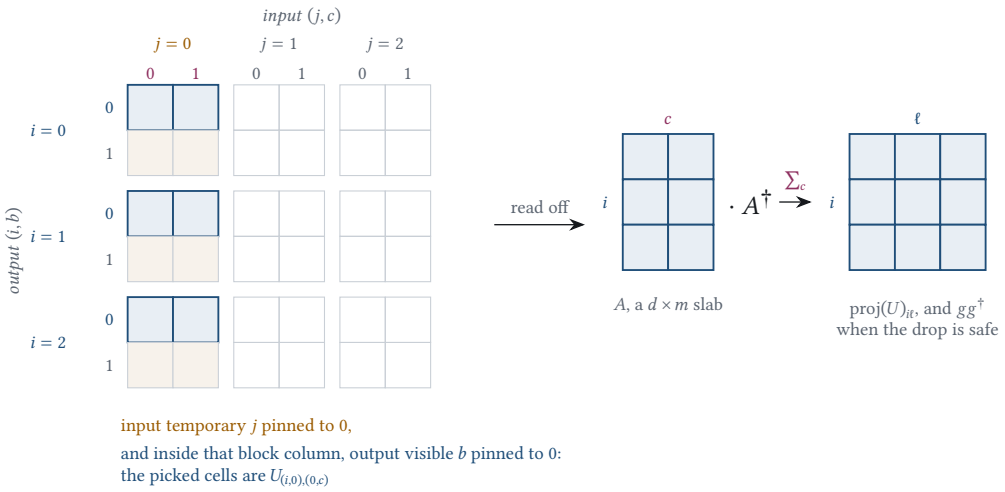


Fig. 3. **Look at the block column**: the allocation pins the input temporary to 0, and inside that column the output visible index is pinned to 0 as well. What is left is a $d \times m$ slab, and $\mathrm{proj}(U)$ is that slab against its own conjugate with the visible index summed away. Drawn at $d = 3$, $m = 2$.

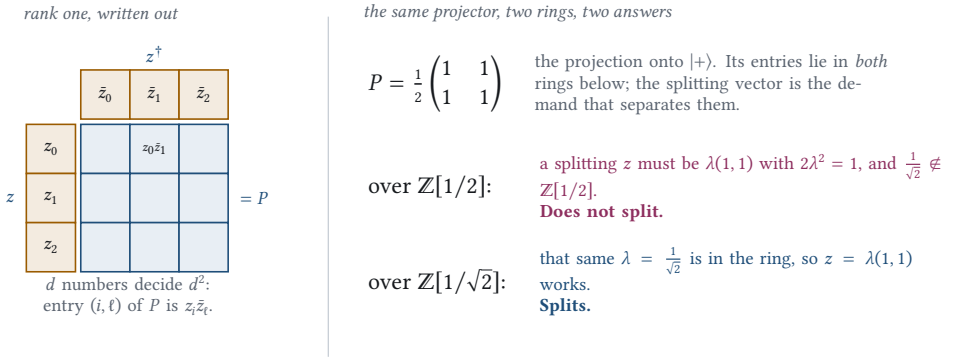


Fig. 4. **Look at the two rows on the right**: the projector is the same matrix in both, and only the ring changes. Splitting asks for a vector over the ring, which is strictly more than asking that the matrix's entries lie in it.

Theorem 5.1 (the relativised criterion). *Let $\mathcal{A}$ be a gate ring completable in the temporary's dimension and U a program over $\mathcal{A}$. Then U is weakenable by gates over $\mathcal{A}$ if and only if U is drop-safe and $\mathcal{A}$ splits* $\mathrm{proj}(U)$.

Figure 5 is this section in one path: repertoire to ring, ring to the splitting test, and the crossed way back.

The proof (Appendix A.2) reads $\mathrm{proj}(U) = gg^\dagger$ off a drop-safe U. A witness over $\mathcal{A}$ prepares g by a gate over $\mathcal{A}$, so the projector splits, and conversely a splitting vector $z = \lambda g$ is completed to a gate by completability, while the visible factor is recovered over $\mathcal{A}$ by contracting U against $\bar{z}$.

Three things in the statement are not the expected shape. *The projector is free*: $\mathrm{proj}(U)$ is a polynomial in the entries of U and their conjugates, so it lies over $\mathcal{A}$, and the side condition is a

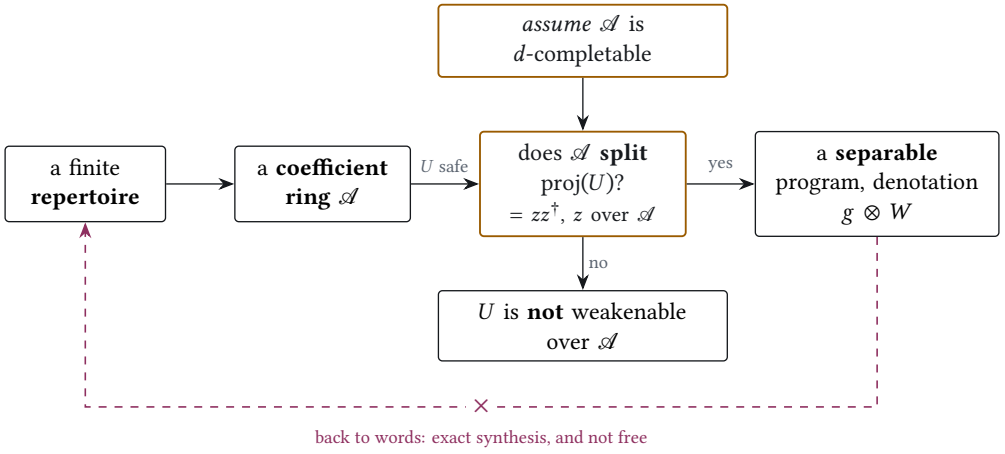


Fig. 5. Which way section 5 runs. **Look at the branch**: splitting is a side condition the ring does not supply, and theorem 5.1 answers it both ways. **Then the dashed arrow**: the way back to words is exact synthesis, and not free—the case that fails is below the theorem.

rank-one factorisation, not a matrix square root. *No clause for the visible factor is needed*, and *the visible register plays no role*, whatever its width. Every gate ring is completable at $d = 2$, and five rings are completable in every dimension, by the column lemmas of the exact-synthesis literature: $\mathcal{R} = \mathbb{Z}[1/\sqrt{2}, i]$, the ring of Clifford+$T$ [13, Lemma 5], and $\mathbb{Z}[1/2]$, $\mathbb{Z}[1/\sqrt{2}]$, $\mathbb{Z}[i/\sqrt{2}]$ and $\mathbb{Z}[1/2, i]$, its integral, real, imaginary and Gaussian restrictions [1, Lemmas 5.4, 5.14, 5.19, 5.25].

Theorem 5.2 (the Clifford+T coefficient ring splits). *Let $\mathcal{R} = \mathbb{Z}[1/\sqrt{2}, i]$ and $d \geq 1$.*

(1) Every rank-one projection on $\mathbb{C}^d$ with entries in $\mathcal{R}$ is $zz^\dagger$ for a vector z over $\mathcal{R}$.

(2) Consequently, at a temporary of any dimension and any visible register, a program over $\mathcal{R}$ is weakenable by gates over $\mathcal{R}$ if and only if it is drop-safe.

The ring $\mathcal{R}$ is exactly the one in which single-qubit Clifford+T operators live [20]: a 2×2 unitary over $\mathcal{R}$ is an ancilla-free Clifford+T word up to a global phase, and conversely. That reading needs the *whole* unitary to be one qubit, temporary and visible together—not merely the visible factor—and a program with a temporary is already past it. Beyond it a gate over $\mathcal{R}$ is a macro-gate, and synthesising one in Clifford+T can need a clean ancilla [13], which a witness reusing the same allocation does not have. Controlled-T is such a case: every ancilla-free two-qubit word in H, T and CNOT has determinant a fourth root of unity, $\det \text{controlled-}T = e^{i\pi/4}$, and the ring's scalars move a two-qubit determinant only by ± 1. The theorem is about the ring, and we do not claim it for a finite repertoire. Part (1) is proved in Appendix A.2 from one classical fact, that $\mathbb{Z}[e^{i\pi/4}]$ has class number one [44, Ch. 11]: a nonzero column of the projection, divided by the greatest common divisor of its entries, has for its squared norm a unit of $\mathbb{Z}[1/\sqrt{2}]$ that is a sum of norms, hence totally positive, hence $|\varepsilon|^2$ for a unit ε of $\mathcal{R}$, and that column divided by ε is z. Part (2) is part (1) inside theorem 5.1, the column lemma supplying completability.

One ring below, the side condition is real. Over $\mathbb{Z}[1/\sqrt{2}]$—the ring of the real Clifford+T gates—the projector of the H-type magic state $\cos\frac{\pi}{8}|0\rangle + \sin\frac{\pi}{8}|1\rangle$ has entries in the ring and does not split, while over $\mathcal{R}$ it does; and a single gate $rot \otimes rot$, for rot the rotation of a qubit by $\pi/8$, realises the separation as a program, drop-safe with that magic state as its final state and weakenable over $\mathcal{R}$

but not over $\mathbb{Z}[1/\sqrt{2}]$. Appendix A.2 computes both. What changed is not the final state but the ring available to name it.

Proposition 5.3 (the restricted rings). *Over $\mathbb{Z}[1/2, i]$ and over $\mathbb{Z}[i/\sqrt{2}]$, every rank-one projection with entries in the ring, in every dimension, is $zz^\dagger$ for a z over the ring. Over $\mathbb{Z}[1/2]$ the projection onto $|+\rangle$ is not, and over $\mathbb{Z}[1/\sqrt{2}]$ the projection onto $\cos\frac{\pi}{8}|0\rangle + \sin\frac{\pi}{8}|1\rangle$ is not.*

The proof (Appendix A.2) is theorem 5.2's argument again: the two complex rings are principal ideal domains whose totally positive real units are norms of units, while over a real ring a splitting vector can only be $\pm g$. Of the five rings named here, the three complex ones split every projection and the two real ones each refuse a named projection.

5.1 What deciding costs

The criterion of definition 3.2 is a finite computation on the denotation, and the denotation is exponentially large in the width. This subsection prices it. Scalars are complex here, so $\Gamma_{ij} = \mathrm{tr}(K_i^\dagger K_j)$. Three definitions are needed first, and they are the only ones.

the input, a circuit program has qubit registers, so $d = 2^k$ and $m = 2^n$, with the temporary allocated at $|0^k\rangle$ and gates on at most three qubits each. A gate is given by its matrix, each entry an algebraic number presented exactly—its minimal polynomial, and an isolating rectangle in binary. The program's *size* counts the gates, the qubits and that presentation.

the leak, $\iota(K)$ is measured on the temporary. Feed the visible register an input ψ, possibly entangled with a reference register, and let $\rho_t(\psi)$ be the temporary's state after the run. Its *impurity* $1-\mathrm{tr}\,\rho_t(\psi)^2$ vanishes exactly when that state is pure, and $\iota(K) = \sup_\psi\big(1 - \mathrm{tr}\,\rho_t(\psi)^2\big)$ is the worst case over inputs.

the leak at one input, $\bar{\iota}(K)$ is the same number read at a single ψ, one whose visible marginal is maximally mixed. That is well defined because $\rho_t(\psi)$ depends on the visible marginal alone, and it is what makes the quantity computable rather than a supremum.

At an input whose visible marginal is maximally mixed—the one $\bar{\iota}$ is measured at—the temporary is in the state $\Gamma^\top/m$, by the entrywise identity $\mathrm{Tr}_v[KK^\dagger] = \Gamma^\top$ (Appendix A.5); so $\bar{\iota}(K) = 1 - \mathrm{tr}(\Gamma^2)/m^2$, and the drop is safe exactly when $\bar{\iota}(K) = 0$, exactly when $\iota(K) = 0$. That is form (4) read physically, and it is what the promise below is stated on.

Theorem 5.4 (the classical price). *Deciding whether a circuit program over the gate set $\{X, \mathrm{CNOT}, \mathrm{Toffoli}\}$ is drop-safe is coNP-complete.*

Membership is two basis inputs that land the temporary in different basis states. Hardness is Bennett's compute–copy–uncompute [2] with the target counted into the temporary: a program that evaluates a Boolean circuit B, copies $B(x)$ into a target and uncomputes its scratch drops the two together safely exactly when B is constant. Constancy is not unsatisfiability, so the reduction emits a fixed unsafe program when $B(0^n) = 1$; what it emits is drop-safe exactly when B is unsatisfiable. Liu et al. show the same bound for a different predicate, whether some unitary can restore the ancilla [28], and that program is restorable for every B (Appendix A.5).

In revision

Proposition 5.5 (the price at the maximally mixed input). *Fix a polynomial $p : \mathbb{N} \to \mathbb{N}$ with $p(n) \geq 1$ for every n, let $\varepsilon = 1/p(\mathit{size})$, and let the circuit program's gates come from a fixed finite set. Deciding, for such a program promised either drop-safe or $\bar{\iota}(K) \geq \varepsilon$, which holds is in BQP.*

The algorithm: run the program twice on independent uniformly random basis inputs and swap-test the two temporaries, which accepts with probability $1 - \bar{\iota}(K)/2$ [4]. The two readings of the leak differ. For $m \geq 2$ Grover's reflection [16] $U = I - 2|w\rangle\langle w|$ has nonzero $+1$ and -1 eigenspaces, and the program that puts the temporary in $|+\rangle$, applies U controlled by it and drops it has $\iota = \frac{1}{2}$, the most a qubit can carry, but $\mathrm{tr}(U)/m = 1 - 2/m$, so $\bar{\iota} = 2/m - 2/m^2$, and the swap test needs order m repetitions to see it, exponentially many in the width. Without a promise gap the verdict is coNP-hard over any gate set containing X, CNOT and Toffoli, and lies in PSPACE when the gate entries come from a fixed number field (Appendix A.5).

Planned On which gate sets is the verdict cheap, and what does a gate one step outside such a set cost? Being written; nothing here is claimed.

6 Measurement, and the lifted calculus

Program (c) needs a language in which a measured outcome becomes a classical parameter. That is a second language, and from here the two are kept apart by name: a *program* is a typed term of definition 3.1, which is what every section up to here was about, and a *command* is a typed c of the calculus just below—which is what (c) becomes on that reading. The rest of the paper is about commands.

The second calculus is two-sorted, in the style of Proto-Quipper's *dynamic lifting* [12]—the Quipper term for reading a measurement outcome into classical control:

$$\begin{aligned} c \;::=\; & \texttt{halt} \mid \texttt{gate}\, U\,; c \mid \texttt{vgate}\, V\,; c \mid \texttt{drop}\, t\,; c \\ & \mid \texttt{let}\ p = \texttt{lift}(\texttt{meas}\, t \in S)\ \texttt{in}\ c \mid \texttt{if}\ e\ \texttt{then}\ c_1\ \texttt{else}\ c_2 \end{aligned}$$

with two contexts: parameters are classical, duplicable and weakenable, and the quantum state is affine, forgettable only by `drop`, after which nothing may name it. There is no allocation form, and none is missing: a command is typed at a flag that starts the temporary live, and the machine starts it at e_{i_0}, so the allocation sits in the judgement rather than in the syntax.

That is what allocFree does for definition 3.1, said the other way round—one temporary, allocated once, before anything runs. One rule connects the two sorts: a lift on a live temporary measures whether it lies in the block S, keeps the surviving branch without renormalising, and binds the answer as a parameter the continuation may test. The *measurement record*, the word of the quantum-trajectory literature [46], is the sequence of which outcome survived at each `lift`, supplied from outside the program. Branch nondeterminism is recovered by quantifying over records.

What a command *does* still has to be said, and the lift is the reason to say it with a machine rather than a function. A lift has two futures and the text does not choose between them; only the record does. So the machine is handed its record from outside, as a tape it consumes one entry per lift; everything else is then determined. A *configuration* $\langle c; \psi; \eta; \sigma \rangle$ carries the command still to run, the joint state ψ, the store η of parameters bound so far, and the record's unread suffix σ.

Two abbreviations make the lift rule readable. A lift on the block S keeps either the part of the temporary inside S or the part outside it, so write

$$P_{\mathsf{in}} = P_S, \qquad P_{\mathsf{out}} = I - P_S, \qquad \beta(\mathsf{in}) = 1, \qquad \beta(\mathsf{out}) = 0$$

for those two projections and for the boolean each of them reports back to the program. Three reductions then carry the weight; a visible gate and a conditional do the obvious thing, and

Appendix A.1 has them.

$$\langle \texttt{gate}\ U\ ; c; \psi; \eta; \sigma\rangle \longrightarrow \langle c; U\psi; \eta; \sigma\rangle,$$
$$\langle \texttt{drop}\ t; c; \psi; \eta; \sigma\rangle \longrightarrow \langle c; \psi; \eta; \sigma\rangle,$$
$$\langle \texttt{let}\ \ p = \texttt{lift}(\texttt{meas}\ t \in S)\ \texttt{in}\ c; \psi; \eta; b \cdot \sigma\rangle \longrightarrow \langle c; (P_b \otimes I)\psi; \eta[p \mapsto \beta(b)]; \sigma\rangle.$$

Read them in order. A gate applies its matrix and moves nothing else. A drop applies *nothing*, which is exactly the point: the temporary is still sitting in ψ, and all that has happened is that the typing has stopped anyone naming it. A lift takes the next entry b off the record, projects the temporary onto that half of the block, does *not* renormalise—so the weight the branch keeps is carried rather than thrown away—and binds $\beta(b)$ into the store for a later conditional to test.

Because the record was fixed before the run, all of this multiplies out into one matrix. Write $E_o(c)$, the *selected denotation* of c along o, for the composite of the gates and block projections the run performs, read off c by a recursion that Appendix A.1 gives together with the typing rules and the remaining reductions; theorem 6.1, just below, is the statement that this matrix and the machine agree. A record is *reachable* for c when $E_o(c) \neq 0$. The drop is *safe under* o when $E_o(c)$ is drop-safe, so an unreachable record is safe. The *weight* of the run at a unit input v is $\|E_o(c)v\|^2$.

Theorem 6.1 (metatheory of the lifted calculus). *Fix a measurement record o.*

(1) A well-typed configuration is terminal or steps; the step preserves typing, the parameter store binds exactly the parameters in scope, and no step names a dropped temporary.

(2) Every run halts, and its terminal joint state is $E_o(c)$ applied to the visible input, where $E_o(c)$ is the selected denotation of c along o.

Hence the drop of c is operationally safe under o if and only if $E_o(c)$ is drop-safe.

The proof, a pair of inductions over commands, is in Appendix A.1. What the theorem buys is that every later statement about a command is a statement about the single linear map $E_o(c)$—and that map is of the kind definition 3.2 is about. The machine starts at $e_{i_0} \otimes v$, so $E_o(c)(v) = E_{o,\varnothing}(c)(e_{i_0} \otimes v)$: an operator on $\mathbb{R}^d \otimes \mathbb{R}^m$ precomposed with the allocation isometry, hence a map $\mathbb{R}^m \to \mathbb{R}^d \otimes \mathbb{R}^m$, which is the type of K. Kraus operators, drop safety and the Gram matrix are therefore read off $E_o(c)$ exactly as off a program's K, one record at a time. Only the normalisation moves: a program's K is an isometry, so $\operatorname{tr}\Gamma = m$, while $E_o(c)$ carries the record's block projections and $\operatorname{tr}\Gamma$ is instead the weight that record keeps, summed over the visible basis. Theorem 6.2 is that bookkeeping.

The two calculi meet at their denotations, not at their grammars. A typed program has no lift, and the gate list its elaboration produces *is* a lift-free command with the same denotation; the lift-free commands are slightly the larger class, since one may apply a visible gate after the drop and no program can write that. So measurement does not break the matrix picture. It indexes it.

6.1 The same verdict, as a channel

Theorem 6.1 paves the way for reasoning with channels: along a record a command is one linear map $E_o(c)$, so the verdict can be stated on the channel that map induces. Fix a measurement record o, write $V_o = E_o(c)$, and build the discard channel of section 3 from V_o instead of from a program's K: $\Phi_o(\rho) = \sum_i K_i \rho K_i^\top$ with K_i the Kraus operators of V_o. Because V_o carries the block projections of the record it is not an isometry, so Φ_o is trace-*non*increasing rather than trace-preserving—a *subchannel*, whose surviving trace is the weight of the branch. The channel is $\sum_o \Phi_o$; the subscript is load-bearing, because without it there is no single branch to speak of. The proportionality lemma of section 3 now does its second job.

Theorem 6.2 (drop safety is Kraus rank one). *Fix a measurement record o. The drop is safe under o if and only if the subchannel Φ_o has a representation with a single Kraus operator. At a unit input v the weight of the run is* $\operatorname{tr}\Phi_o(vv^\top)$.

Proof. $\operatorname{tr}\Phi_o(vv^\top) = \sum_i \|K_i v\|^2 = \|V_o v\|^2$ is the weight. If the drop is safe, $K_i = c_i W$ and the sum collapses to $\Phi_o(\rho) = (cW)\rho(cW)^\top$ with $c^2 = \sum_i c_i^2$. Conversely, if $\Phi_o(\rho) = M\rho M^\top$ for every ρ, then at $\rho = xx^\top$ the hypothesis of lemma 3.5 holds, so $K_i = c_i M$ and $V_o = (c_i)_i \otimes M$ is drop-safe. Nothing assumed V_o isometric, and the zero map has the single Kraus operator 0, so an unreachable record is safe here too. □

One caution, and it is why the paper is indexed by measurement records rather than by channels. Two well-typed commands on a temporary qubit and a visible qubit can have the same summed channel $\Phi_0 + \Phi_1$ and the same weight on every branch at every input, with opposite verdicts on every branch: H then CZ then a lift, against a lift then a controlled write, both summing to the dephasing channel (Appendix A.1). So an analysis that reads the channel a program implements, or samples its statistics, cannot decide the drop. One that reads the program can, and its instance is the calculus above.

7 The boundary of one verdict per command

Planned What a verdict that does not read the measurement record must give up, and where that boundary starts. Not an undecidability result: safety under every reachable record is decided by running theorem 4.2's rank test once per record. Being written; nothing here is claimed.

8 The repair: a check the run can answer

Planned What a compiler can emit when no verdict fixed before the run will do: a table with one row per measurement record, computed before the run and looked up after it, which fig. 6 draws. The rule at one lift, the two hypotheses it needs, and the closed form for a run of consecutive lifts are being written; nothing here is claimed.

9 Several temporaries, and lifetimes

Planned The criterion is stated for one temporary. What a summary must retain when a program allocates several—composed into one, tracked a lifetime at a time as a Rust-style region discipline would [19], or crossed by a procedure boundary—is being written; section 10's borrowed-ancilla row reports only what the checker printed.

10 Implementation and evaluation

In revision The preceding sections prove; this one runs what they define. The artifact is one Haskell package with two implementations of one question. `DQT` builds the denotation K as a matrix, forms the Kraus operators, and takes the exact rank of their $d \times m^2$ reshaping (definition 3.2), a cost exponential in the width. `S`, the surface language of the corpus, never builds K: it runs the program on dim +1 test inputs, the visible basis plus all-ones, factors the state at the drop site, and asks whether the resulting lines agree. A bridge module compares the two, and **they agree on every catalogue verdict**; each engine's runtime check is pinned against its own criterion. They share no code but one exact cyclotomic backend, so the comparison varies the algorithm

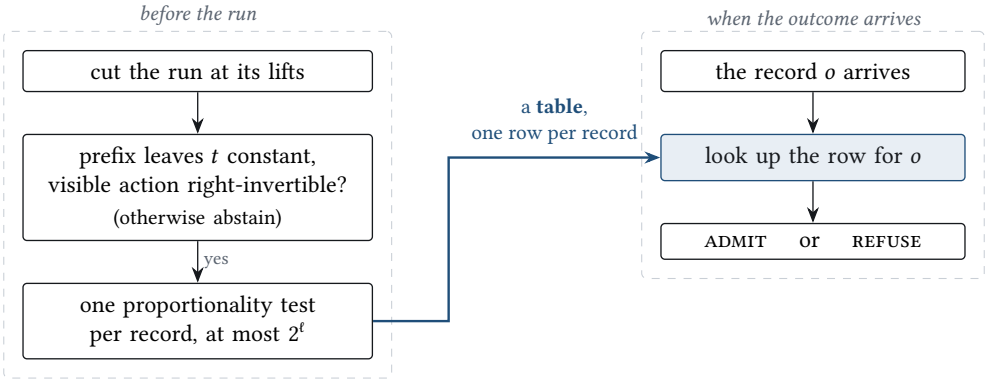


Fig. 6. The two lanes of section 8. **Look at the arrow between them**: what the compiler emits is a *table*, one row per measurement record, not a verdict. The left lane is the work section 7 leaves open before the run, and it abstains rather than guessing when the prefix does not leave the temporary constant with a right-invertible visible action. The right lane needs the run, and it is a lookup.

and holds the arithmetic fixed. A companion Lean library over Mathlib mechanises theorem 4.2 from the calculus's syntax and typing rules, and proves the Gram identity, the temporary's state at the maximally mixed visible input, that local gates compose and carry a product input to a product output, and, in abstract form, the constancy lemma that section 7 will state. The Haskell suite runs 58 named statements in the theory layer and 244 checks in the language layer over seven of the table's eight algorithms and three more (Shor's order finding, Simon's problem, a temporary AND uncomputed by measurement), the C^3X borrow being pinned by three checks on its source files.

10.1 The catalogue: a producer rule against the criterion

In revision Seventeen small programs, each built to make one point and listed in the supplement, Appendix B, run against two rules. **R1** accepts a drop when every gate is a permutation, the gate-level reading of "creates no superposition". **R2** adds "and no gate writes visible data into the temporary", the shape of Silq's producer rule. The criterion accepts **twelve** of the seventeen. R1 accepts four, one wrongly: `compute-alone` creates no superposition and still leaks, because it writes visible data into the temporary. R2 is sound and accepts **two** of the twelve. Deutsch–Jozsa, Bernstein–Vazirani, Grover's oracle and the two-gate program of section 1 are safe and refused by it.

10.2 Eight textbook algorithms

In revision We write eight textbook algorithms in `S`, compute the verdict at every `drop`, and run the runtime check and Silq on each row's smallest faithful program. Two rows leave the calculus: C^3X takes its temporary on a state it did not prepare, and superdense coding lifts a *visible* register while the temporary is live. Their verdicts are the checker's, computed on the same denotation by the same test, and no theorem here covers them. The Silq column is transcripts against two source builds, `a47d974` and `5417dce`; `code/silq/TRANSCRIPT.md` says which run used which, and marks the one attribution it had to infer. The Q# column is its release requirement,

unenforced on hardware.

algorithm	this theory says	Silq, run	Q# release
Deutsch–Jozsa	safe, left in $[\|-\rangle]$ (section 4.2)	*cannot synthesize forget*	fails at $\|-\rangle$
Bernstein–Vazirani	safe, $W = X^s$	the same error	fails at $\|-\rangle$
Grover oracle, compute–uncompute	safe, left in $[\|0\rangle]$	accepts, runs	passes
C^3X, borrowed dirty ancilla	safe on the two loans run, left on the loan's own ray	expressible, unchecked	promise, unchecked
Phase estimation	safe, left on the eigenstate line	the same error	fails at $\|u\rangle$
Superdense coding	safe under every record	accepts, no drop	passes
Teleportation	both drops safe, every record, for the message $\|+\rangle$	accepts, runs	runtime only
Repeat-until-success round	*outcome-dependent*, the runtime check decides both	accepts, then *bad forget* on 6/20 seeds	no single release ray

The disagreements never go the dangerous way, but "no disagreement" is often "no ruling". In these eight rows Silq never accepts an automatic `drop` that this theory refuses, and it refuses three safe ones; only the Grover row is positive agreement, since superdense coding's encoding has no `forget`, the borrowed ancilla is unchecked, and teleportation's survivor is the receiver's register. Containing a measurement is not the problem: superdense coding and teleportation lift outcomes and are not outcome-dependent. On the last row Silq accepts a hand-supplied value witness and then fails at run time, *bad forget*, on 6 of 20 seeds. That refutes the one value we supplied, and we searched for no other; that no fixed value can serve at all is what section 7 will show. Phase estimation is complex, so its verdict is taken up to phase, on the ray the temporary is left on rather than on a real vector. These rows measure agreement and coverage, not scale: both procedures are exponential in register width.

The C^3X row is a program we did not write: Gidney's four-Toffoli 3-controlled X with one *dirty* ancilla, the first case study of Su et al.'s heap logic [37]. In the surface language:

```
vis q1:2, q2:2, q3:2, q:2; newdirty a:2 @ (1,0);   -- the loan: a arrives on |0>
gate CCX q3 a q; gate CCX q1 q2 a;                 -- two Toffolis
gate CCX q3 a q; gate CCX q1 q2 a; drop a;         -- again, then return the loan
```

Our checker admits the drop, printing the loan's own ray $(1, 0)$ and, with the loan $|+\rangle$, that loan's ray. What it does not do is certify the borrow for *every* lent state: that needs a finite test set, and this document does not state one.

11 Related work

In revision *Affine typing and circuit languages.* Selinger and Valiron make quantum data affine, so discard is a typing restriction rather than a question [35], and QWIRE supplies the circuit/host split [33]. Twist and Guppy address the neighbouring obligation that a value be unentangled from what it does not own [22, 23, 49]. Twist comes closest: its Corollary 6.2 is a semantic *iff* for purity, weakened in §7.6 to a sufficient condition once assertions are added, and its checker runs a static-plus-runtime split, but it has no completeness theorem and no proof that the retreat is forced (theorem 4.2 and section 7).

Automatic uncomputation. Silq [3] and Unqomp [31] ask how a value was produced (section 1). Cavy [29] and Qurts [18] track ancilla lifetimes; Cavy states the obligation semantically—a short-lived ancilla returns *unentangled*—and pursues it by lifetime-directed uncomputation, its correctness proof left open. Qunity [43], Reqomp [32] and modular synthesis [42] work under

error, space or purity restrictions. ReQWIRE frames the gap: a semantic characterisation of ancilla validity, then syntactic conditions it calls sufficient but not necessary [34].

The nearest neighbour. Qutes gives scope-bounded, liveness-guided uncomputation a semantic account—semantic lifetimes, a Restoration Invariant, nested scopes—and its boundary is exact where it applies: for a temporary entangled with a variable that stays live across the boundary, restoration is possible *if and only if* the subcomputation is *output-isolable* with respect to that variable, decomposing as $U_L \circ V_T$ with V_T acting on the temporary alone [10, Def. 5, Lem. 11]; a companion HPDC paper studies the structural parallelism such closed regions create [11]. Read literally, Definition 5 puts the two factors on disjoint registers, so the subcomputation denotes a temporary factor tensored with a visible one—both gates *local* in the sense of section 3—and that is strictly stronger than drop safety; their worked example inserts the temporary's own history, reading a live input variable, so we do not read it as a comparison of strength. Reclamation returns the temporary to its *initial* state [10, Def. 6], and our question is the other one: may a temporary be dropped *without* restoration? Deutsch–Jozsa's ancilla ends on $|-\rangle$, not where it was allocated, while theorem 4.2 asks only that it end *somewhere* fixed. Where a measurement outcome influences what follows their invariant cannot hold past that point [10, Lem. 9]; that is section 7's regime, where the verdict is indexed by the record rather than refused.

Verification logics. Q* makes separability an assertion justifying discard [17]; Tsai et al. compile such assertions to automata [41]. Su et al.'s quantum BI logic is sound and relatively complete for dirty-allocation heaps [37]. Hoare Meets Heisenberg certifies disposal for a stabilizer fragment [38]. None contradicts section 7: each verifies a given postcondition, our static ceiling; the impossible boolean is the run-sensitive one, its sound side Chen et al.'s dead-measurement analysis [8].

Exact conditions at circuit level. Li et al. verify that a circuit restores its clean or dirty ancillas, by commutation checks on decision diagrams and model counting, and repair faults [27]. Liu et al. show that deciding whether an uncomputation exists is coNP-complete for reversible Boolean circuits over $\{\mathrm{CCNOT}, X\}$, and coNP-hard wherever those two are polynomial-size constructible [28]. Su et al. give finite tests for the identity-on-the-borrowed-qubit condition, two states for classical circuits [36], and Bona assumes safe dirty-ancilla blocks—identity on the borrowed qubit—and schedules semantics-preserving borrow substitutions [47]. All four work on fixed unitary circuits. A drop asks whether restoration is needed at all, strictly between restoration and restorability (section 1) and, with measurement, dependent on the record (section 7). Section 5.1 prices the verdict itself [2, 40].

The reversible lineage. In the Π family a fractional-type token reclaims allocated storage [7], and de-allocation is a runtime value match [6]. QuantumPi adds quantum ancillas, hiding and measurement [5], but no outcome-indexed certificate. Our calculus is not Π: on outcome-dependent commands the runtime position is forced, and the fractional reuse rule is sound and strictly weaker (section 4). The spooky pebble game [24] measures rather than restores the ancilla, and its uncomputed ancilla is drop safety at $g = |0\rangle$—restoration again—with the phase left as the continuation's obligation.

12 Conclusion, scope, and what is open

In revision A quantum temporary may be dropped when the rest of the program cannot tell. For a measurement-free typed program three conditions coincide—weakenable, drop-safe, contextually indistinguishable from a separable program—and the test is proportionality of the Kraus operators: Kraus rank one, equivalently a pure temporary at the maximally mixed visible input. Over a gate ring completable in the temporary's dimension it acquires one side condition, supplied by the Clifford+T ring and its two complex restrictions and not by its two real ones. Once a

command lifts an outcome the condition survives and the *verdict* does not: one boolean fixed before the run cannot be right on both records, so a sound one must refuse a safe run, on exactly the outcome-dependent commands. The missing argument is the measurement record, so the verdict must become record-aware; ours is a table the compiler emits, one row per record. Every verdict here is defined exactly from a selected denotation and computed in exact arithmetic; proposition 5.5 alone is a bounded-error promise test. Open: a gate ring with no column lemma, the positive side of the per-lifetime obstruction, and whether the runtime check of section 8 is exactly some set of typing rules. Five boxes marked *Planned* name the results this draft does not yet carry, each being written.

References

[1] Matthew Amy, Andrew N. Glaudell, and Neil J. Ross. 2020. Number-Theoretic Characterizations of Some Restricted Clifford+T Circuits. *Quantum* 4, Article 252 (2020). doi:10.22331/q-2020-04-06-252

[2] Charles H. Bennett. 1973. Logical Reversibility of Computation. *IBM Journal of Research and Development* 17, 6 (1973), 525–532. doi:10.1147/rd.176.0525

[3] Benjamin Bichsel, Maximilian Baader, Timon Gehr, and Martin Vechev. 2020. Silq: A High-Level Quantum Language with Safe Uncomputation and Intuitive Semantics. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation*. 286–300. doi:10.1145/3385412.3386007

[4] Harry Buhrman, Richard Cleve, John Watrous, and Ronald de Wolf. 2001. Quantum Fingerprinting. *Physical Review Letters* 87 (2001), 167902. doi:10.1103/PhysRevLett.87.167902

[5] Jacques Carette, Chris Heunen, Robin Kaarsgaard, and Amr Sabry. 2024. The Quantum Effect: A Recipe for QuantumPi. *Proceedings of the ACM on Programming Languages* (2024). doi:10.1145/3674625

[6] Chao-Hong Chen. 2021. *Algebraic Information Effects*. Ph. D. Dissertation. Indiana University.

[7] Chao-Hong Chen, Vikraman Choudhury, Jacques Carette, and Amr Sabry. 2020. Fractional Types: Expressive and Safe Space Management for Ancilla Bits. In *Reversible Computation — 12th International Conference, RC 2020 (Lecture Notes in Computer Science, Vol. 12227)*. Springer, 169–186. doi:10.1007/978-3-030-52482-1_10

[8] Yanbin Chen, Qunyou Liu, Yu Wang, Christian B. Mendl, and Helmut Seidl. 2026. GPU-Accelerated Host-Aware Dead-Measurement Detection in Hybrid Quantum–Classical Programs. arXiv:2607.09976 [quant-ph] To appear at SAS 2026; full version.

[9] Man-Duen Choi. 1975. Completely Positive Linear Maps on Complex Matrices. *Linear Algebra Appl.* 10, 3 (1975), 285–290. doi:10.1016/0024-3795(75)90075-0

[10] Simone Faro, Francesco Pio Marino, and Gabriele Messina. 2026. Reversible Lifetime Semantics for Quantum Programs. arXiv:2603.14538 [cs.PL]

[11] Simone Faro, Francesco Pio Marino, and Gabriele Messina. 2026. Structural Parallelism in Quantum Programs. In *Proceedings of the 35th International Symposium on High-Performance Parallel and Distributed Computing (HPDC '26)*. ACM. doi:10.1145/3806645.3816162 Builds on the Qutes lifetime discipline; Theorem 3.3 restates the output-isolable decomposition $U = U_L \circ V_t$..

[12] Peng Fu, Kohei Kishida, Neil J. Ross, and Peter Selinger. 2023. Proto-Quipper with Dynamic Lifting. *Proceedings of the ACM on Programming Languages* 7, POPL (2023). doi:10.1145/3571204

[13] Brett Giles and Peter Selinger. 2013. Exact Synthesis of Multiqubit Clifford+T Circuits. *Physical Review A* 87, 3 (2013), 032332. doi:10.1103/PhysRevA.87.032332 arXiv:1212.0506; Lemma 5 is the column lemma.

[14] Joseph A. Goguen and José Meseguer. 1982. Security Policies and Security Models. In *1982 IEEE Symposium on Security and Privacy*. doi:10.1109/SP.1982.10014

[15] Alexander S. Green, Peter LeFanu Lumsdaine, Neil J. Ross, Peter Selinger, and Benoît Valiron. 2013. Quipper: A Scalable Quantum Programming Language. *ACM SIGPLAN Notices (PLDI)* 48, 6 (2013), 333–342. doi:10.1145/2499370.2462177

[16] Lov K. Grover. 1996. A Fast Quantum Mechanical Algorithm for Database Search. In *Proceedings of the Twenty-Eighth Annual ACM Symposium on the Theory of Computing (STOC)*. ACM, 212–219. doi:10.1145/237814.237866

[17] Kesha Hietala, Sarah Marshall, Robert Rand, and Nikhil Swamy. 2022. Q*: Implementing Quantum Separation Logic in F*. In *Third International Workshop on Programming Languages for Quantum Computing*. https://rand.cs.uchicago.edu/files/planqc_2022.pdf

[18] Kengo Hirata and Chris Heunen. 2025. Qurts: Automatic Quantum Uncomputation by Affine Types with Lifetime. *Proceedings of the ACM on Programming Languages* 9, POPL, Article 6 (2025), 155–182 pages. doi:10.1145/3704842

[19] Steve Klabnik and Carol Nichols. 2023. *The Rust Programming Language* (2 ed.). No Starch Press. Ch. 15.5; the quoted sentence was checked against the online edition, doc.rust-lang.org/book/ch15-05-interior-mutability.html, 2026-09-05.

[20] Vadym Kliuchnikov, Dmitri Maslov, and Michele Mosca. 2013. Fast and Efficient Exact Synthesis of Single-Qubit Unitaries Generated by Clifford and T Gates. *Quantum Information and Computation* 13, 7–8 (2013), 607–630. doi:10.26421/QIC13.7-8-4

[21] Emanuel Knill and Raymond Laflamme. 1997. Theory of Quantum Error-Correcting Codes. *Physical Review A* 55, 2 (1997), 900–911. doi:10.1103/PhysRevA.55.900

[22] Mark Koch, Agustín Borgna, Craig Roy, Alan Lawrence, Kartik Singhal, Seyon Sivarajah, and Ross Duncan. 2025. Imperative Quantum Programming with Ownership and Borrowing in Guppy. arXiv:2510.13082 [cs.PL] Presented at PLanQC 2025.

[23] Mark Koch, Alan Lawrence, Kartik Singhal, Seyon Sivarajah, and Ross Duncan. 2025. GUPPY: Pythonic Quantum-Classical Programming. arXiv:2510.12582

[24] Niels Kornerup, Jonathan Sadun, and David Soloveichik. 2025. Tight Bounds on the Spooky Pebble Game: Recycling Qubits with Measurements. *Quantum* 9 (2025), 1636. arXiv:2110.08973 [quant-ph] doi:10.22331/q-2025-02-18-1636

[25] Karl Kraus. 1983. *States, Effects, and Operations: Fundamental Notions of Quantum Theory*. Lecture Notes in Physics, Vol. 190. Springer, Berlin, Heidelberg. doi:10.1007/3-540-12732-1

[26] Dennis Kretschmann, Dirk Schlingemann, and Reinhard F. Werner. 2008. The Information-Disturbance Tradeoff and the Continuity of Stinespring's Representation. *IEEE Transactions on Information Theory* 54, 4 (2008), 1708–1717. doi:10.1109/TIT.2008.917696

[27] Jiqi Li, Jingyi Mei, Wang Fang, and Ji Guan. 2026. Formal Verification of Quantum Ancilla Safety. In *Computer Aided Verification*. Springer Nature Switzerland, 326–348. doi:10.1007/978-3-032-32537-2_16 arXiv:2608.13099.

[28] Chenke Liu, Li Zhou, and Boning Meng. 2026. Quantum Uncomputation of Clean and Dirty Ancilla Qubits. arXiv:2608.09578

[29] Christopher Michael McNally. 2021. *Practical Modern Quantum Programming*. S.M. thesis. Massachusetts Institute of Technology. https://inspirehep.net/files/aa1f237c1d580cc362cf2bf89f5c6701

[30] Microsoft. 2025. Quantum Memory Management in Q#. Microsoft Learn, Azure Quantum documentation. https://learn.microsoft.com/en-us/azure/quantum/user-guide/language/statements/quantummemorymanagement

[31] Anouk Paradis, Benjamin Bichsel, Samuel Steffen, and Martin Vechev. 2021. Unqomp: Synthesizing Uncomputation in Quantum Circuits. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI '21)*. ACM, 222–236. doi:10.1145/3453483.3454040

[32] Anouk Paradis, Benjamin Bichsel, and Martin Vechev. 2024. Reqomp: Space-Constrained Uncomputation for Quantum Circuits. *Quantum* 8 (2024), 1258. doi:10.22331/q-2024-02-19-1258

[33] Jennifer Paykin, Robert Rand, and Steve Zdancewic. 2017. QWIRE: A Core Language for Quantum Circuits. In *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages (POPL '17)*. 846–858. doi:10.1145/3009837.3009894

[34] Robert Rand, Jennifer Paykin, Dong-Ho Lee, and Steve Zdancewic. 2019. ReQWIRE: Reasoning about Reversible Quantum Circuits. *Electronic Proceedings in Theoretical Computer Science* 287 (2019), 299–312. doi:10.4204/EPTCS.287.17

[35] Peter Selinger and Benoît Valiron. 2006. A Lambda Calculus for Quantum Computation with Classical Control. *Mathematical Structures in Computer Science* 16, 3 (2006), 527–552. doi:10.1017/S0960129506005238

[36] Bonan Su, Li Zhou, Yuan Feng, and Mingsheng Ying. 2025. Borrowing Dirty Qubits in Quantum Programs. arXiv:2508.17190 [quant-ph]

[37] Bonan Su, Li Zhou, Yuan Feng, and Mingsheng Ying. 2026. An Interpretation of Bunched Logic for Reasoning about Heap-Manipulating Quantum Programs. arXiv:2409.10153 [cs.PL] arXiv v2, 5 Aug 2026; v1 (16 Sep 2024) was titled "BI-based Reasoning about Quantum Programs with Heap Manipulations".

[38] Aarthi Sundaram, Robert Rand, Kartik Singhal, Youngchan Cho, and Brad Lackey. 2026. Hoare Meets Heisenberg: A Lightweight Logic for Quantum Programs. *Quantum* 10 (2026), 2172. doi:10.22331/q-2026-07-23-2172

[39] Krysta Svore, Alan Geller, Matthias Troyer, John Azariah, Christopher Granade, Bettina Heim, Vadym Kliuchnikov, Mariia Mykhailova, Andres Paz, and Martin Roetteler. 2018. Q#: Enabling Scalable Quantum Computing and Development with a High-level DSL. In *Proceedings of the Real World Domain Specific Languages Workshop 2018 (RWDSL2018)*. Article 7, 10 pages. doi:10.1145/3183895.3183901

[40] Yu Tanaka. 2009. Exact Non-Identity Check Is NQP-Complete. arXiv:0903.0675 [quant-ph]

[41] Wei-Lun Tsai, Yu-Fang Chen, and Ondřej Lengál. 2026. A Practical Specification Language for Automatic Quantum Program Verification. arXiv:2605.05786 [cs.LO]

[42] Hristo Venev, Timon Gehr, Dimitar Dimitrov, and Martin Vechev. 2024. Modular Synthesis of Efficient Quantum Uncomputation. *Proceedings of the ACM on Programming Languages* 8, OOPSLA2, Article 345 (2024), 2097–2124 pages. doi:10.1145/3689785

[43] Finn Voichick, Liyi Li, Robert Rand, and Michael Hicks. 2023. Qunity: A Unified Language for Quantum and Classical Computing. *Proceedings of the ACM on Programming Languages* 7, POPL (2023), 921–951. doi:10.1145/3571225

[44] Lawrence C. Washington. 1997. *Introduction to Cyclotomic Fields* (2 ed.). Graduate Texts in Mathematics, Vol. 83. Springer. doi:10.1007/978-1-4612-1934-7

[45] John Watrous. 2018. *The Theory of Quantum Information.* Cambridge University Press. doi:10.1017/9781316848142 Theorem 2.22 and Corollaries 2.23, 2.27 for the operator-sum form and its freedom; the Choi rank is defined on p. 78 and isometric channels on p. 91.

[46] Howard M. Wiseman and Gerard J. Milburn. 2010. *Quantum Measurement and Control.* Cambridge University Press. doi:10.1017/CBO9780511813948

[47] Xiaoquan Xu, Chenke Liu, Boning Meng, Zihao Shen, and Li Zhou. 2026. Bona: Automatic Management of Dirty Ancilla Borrowing in Quantum Circuits. arXiv:2608.08765 [cs.PL]

[48] Mingsheng Ying, Yuan Feng, and Nengkun Yu. 2013. Quantum Information-Flow Security: Noninterference and Access Control. In *2013 IEEE 26th Computer Security Foundations Symposium.* 130–144. doi:10.1109/CSF.2013.16

[49] Charles Yuan, Christopher McNally, and Michael Carbin. 2022. Twist: Sound Reasoning for Purity and Entanglement in Quantum Programs. *Proceedings of the ACM on Programming Languages* 6, POPL, Article 30 (2022), 32 pages. doi:10.1145/3498691